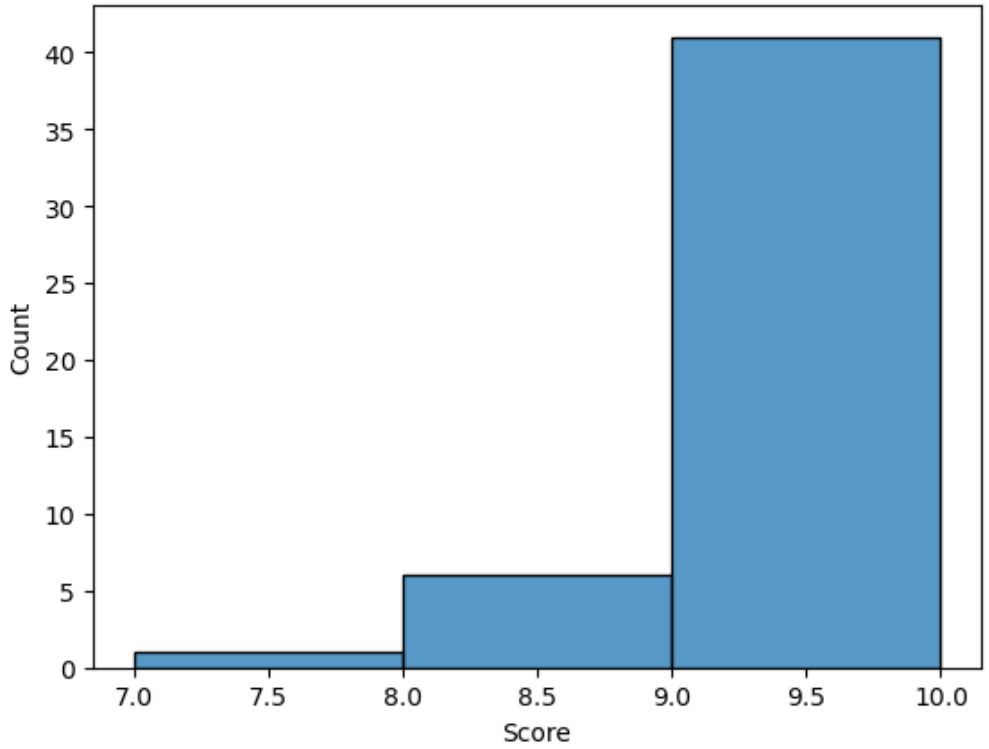
EVE310 Lab 3

September 18, 2025

Lab 2 Quiz Recap

Mean: 9.375
Mode: 10



Lab 2 Quiz Recap

8/28/12 0:00	138.091822	104.174532	87.5	136.312679	102.736076	1	wk	Summer
8/29/12 0:00	136.312679	102.736076	88.5	NaN	102.736076	1	wk	Summer
8/30/12 0:00	NaN	97.3607912	90.5	NaN	102.736076	1	wk	Summer
8/31/12 0:00	NaN	103.606721	86.5	168.488679	102.736076	1	wk	Summer
9/1/12 0:00	168.488679	82.9383723	89.5	79.4936476	116.249996	1	wknd	Fall
9/2/12 0:00	79.4936476	111.972481	87.5	99.5941842	116.249996	1	wknd	Fall
9/3/12 0:00	99.5941842	138.091822	88	132.943662	116.249996	1	wk	Fall

```
In [8]: np_min = np.min(water_con)
        np_max = np.max(water_con)

        py_min = min(water_con)
        py_max = max(water_con)

        np_nanmin = np.nanmin(water_con)
        np_nanmax = np.nanmax(water_con)

        print('np.min:',np_min)
        print('np.max:',np_max)

        print('py min:',py_min)
        print('py max:',py_max)

        print('np.nanmin:',np_nanmin)
        print('np.nanmax:',np_nanmax)

np.min: 2.536225899
np.max: 148.6909751
py min: 0.037854118
py max: 188.2106742
np.nanmin: 0.037854118
np.nanmax: 188.2106742
```

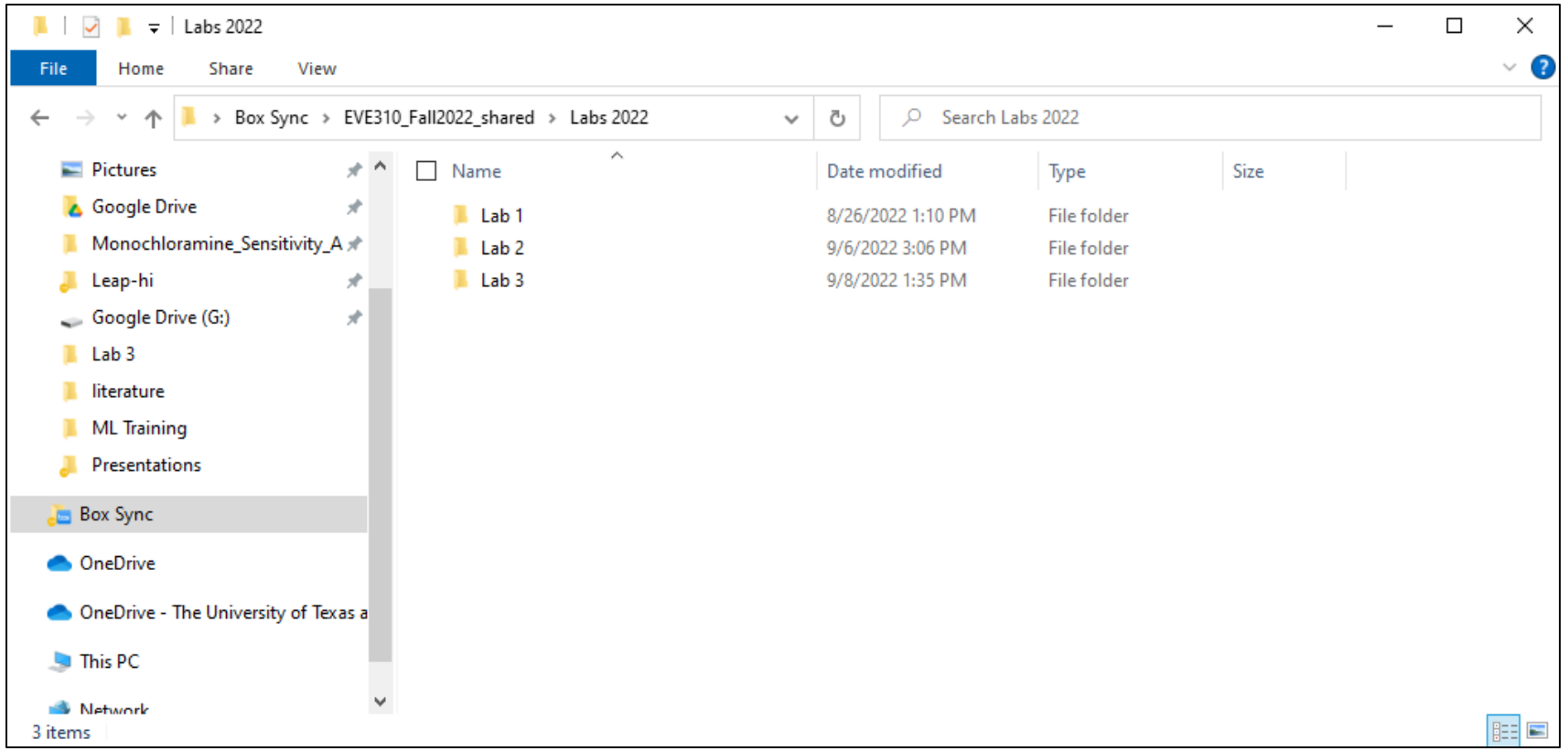
Incorrect
Correct
Correct

- NaN: very common in real world data
- np.min and np.max do not work with arrays with NaN values
- min() and max() do not work if array starts with NaN values
- **Instead, use:**
  - np.nanmin() and np.nanmin() are the most robust functions to handle NaNs
- We used np.min and np.max with this data, so our answers were incorrect!
- Always inspect your data and spot check your work

More to come on data cleaning this semester!

File Management Suggestion

- Make a folder for this class, then a separate folder for each lab



Learning objectives

- Distinction between arrays and dataframes
- Size and Shape
- Summarizing a dataframe
- Renaming, removing, and adding columns of a dataframe
- Indexing a dataframe

Follow along with the Lab 3 Jupyter Notebook tutorial posted Canvas. Note the tutorial uses the same dataset as last week!

Differences and similarities between Arrays and Dataframes

	Array	Dataframe
Package	Numpy	Pandas
Data Type	Homogeneous (numerical)	Heterogenous (numbers or strings, but each column bust be the same type)
Number of Dimensions	Higher-dimensional arrays available (3D, 4D, etc)	2D at maximum
Indexing	Row and column numbers	Row and column numbers, or row and column names
Description of Data	None	Row and Column names embedded in python variable
Additional Notes	Useful for high-performance algebraic calculations	Useful for working with recorded data in a tabular structure

The Anatomy of a Dataframe

Rows: Individual observations

Columns: Parameter being measured or described (also sometimes called attributes, or features)

	A	B	C	D	E	F	G	H	I
1	allDate	prevDay	prevWeek	avgTemp	wF	MonMed	classSessio	dayType	season
2	1/14/2009 0:00	NaN	NaN	47	7.192282	10.78842	0	wk	Winter
3	1/15/2009 0:00	7.192282	NaN	46	6.813741	10.78842	0	wk	Winter
4	1/16/2009 0:00	6.813741	NaN	41.5	7.570824	10.78842	0	wk	Winter
5	1/17/2009 0:00	7.570824	NaN	57.5	5.678118	10.78842	0	wknd	Winter
6	1/18/2009 0:00	5.678118	NaN	61	6.4352	10.78842	0	wknd	Winter


```
#Import the data into python as a dataframe from the csv file
water_df = pd.read_csv('JES_Water.csv')

#Take a look at the first 5 rows of the dataframe using the head function
water_df.head(5)
```

	allDate	prevDay	prevWeek	avgTemp	wF	MonMed	classSession	dayType	season
0	1/14/2009 0:00	NaN	NaN	47.0	7.192282	10.788424	0	wk	Winter
1	1/15/2009 0:00	7.192282	NaN	46.0	6.813741	10.788424	0	wk	Winter
2	1/16/2009 0:00	6.813741	NaN	41.5	7.570824	10.788424	0	wk	Winter
3	1/17/2009 0:00	7.570824	NaN	57.5	5.678118	10.788424	0	wknd	Winter
4	1/18/2009 0:00	5.678118	NaN	61.0	6.435200	10.788424	0	wknd	Winter

Head of a dataframe

head: shows the first few rows of a dataframe

The number of rows is specified by the number in parenthesis.

```
#Import the data into python as a dataframe from the csv file
water_df = pd.read_csv('JES_Water.csv')

#Take a look at the first 5 rows of the dataframe using the head function
water_df.head(5)
```

	allDate	prevDay	prevWeek	avgTemp	wF	MonMed	classSession	dayType	season
0	1/14/2009 0:00	NaN	NaN	47.0	7.192282	10.788424	0	wk	Winter
1	1/15/2009 0:00	7.192282	NaN	46.0	6.813741	10.788424	0	wk	Winter
2	1/16/2009 0:00	6.813741	NaN	41.5	7.570824	10.788424	0	wk	Winter
3	1/17/2009 0:00	7.570824	NaN	57.5	5.678118	10.788424	0	wknd	Winter
4	1/18/2009 0:00	5.678118	NaN	61.0	6.435200	10.788424	0	wknd	Winter

Use a “print” statement around this in Spyder

```
#Note: in jupyter, the above command works. But in spyder, use:
print(water_df.head(5))
```

Data Types in Pandas

Pandas dtype	Python type	NumPy type	Usage
object	str or mixed	string_, unicode_, mixed types	Text or mixed numeric and non-numeric values
int64	int	int_, int8, int16, int32, int64, uint8, uint16, uint32, uint64	Integer numbers
float64	float	float_, float16, float32, float64	Floating point numbers
bool	bool	bool_	True/False values
datetime64	NA	datetime64[ns]	Date and time values
timedelta[ns]	NA	NA	Differences between two datetimes
category	NA	NA	Finite list of text values

Overview of data types:
https://pbpython.com/pandas_dtypes.html
https://stackoverflow.com/questions/43440821/the-real-difference-between-float32-and-float64

How big is this dataset? How many datapoints does it have? How many rows? How many columns?

Size and Shape of a dataframe

size: total number of individual values stored in the dataframe

shape: number of rows and columns in a dataframe

```
#size
siz=water_df.size
print('The size of the dataframe is', siz)
#shape
[row,col]=water_df.shape
print('The number of rows in the dataframe is',row)
print('The number of columns in the dataframe is',col)


print('The number of rows multiplied by the number of columns is',row*col)
```

```
The size of the dataframe is 27846
The number of rows in the dataframe is 3094
The number of columns in the dataframe is 9
The number of rows multiplied by the number of columns is 27846
```

Summary Statistics

Describe

describe: obtain summary statistics of each column in a dataframe

Useful for doing spot-checks to make sure the data looks like you would expect.

```
1  # Obtain summary statistics from each column on the dataframe
2  water_df.describe()
```

	prevDay	prevWeek	avgTemp	wF	MonMed	classSession
count	3034.000000	3028.000000	3094.000000	3035.000000	3094.000000	3094.000000
mean	78.608096	78.636982	70.554299	78.599382	82.136933	0.604719
std	50.591626	50.635127	14.474940	50.585565	41.708586	0.488990
min	0.037854	0.037854	21.500000	0.037854	5.659191	0.000000
25%	36.084438	36.065511	60.000000	36.093901	51.103059	0.000000
50%	75.708236	75.708236	72.500000	75.708236	76.389610	1.000000
75%	121.133177	121.133177	83.000000	121.133177	117.082787	1.000000
max	188.210674	188.210674	96.000000	188.210674	170.305676	1.000000

Use a “print” statement around this in Spyder

```
1  #Note: in jupyter, the above command works. But in spyder, use:
2  print(water_df.describe())
```

Describe: spot checking data

```
1  # Obtain summary statistics from each column on the dataframe
2  water_df.describe()
```

Count is not the same in every column, it takes into consideration how many NaN values there are in each column

The min and max wF here are correct

Mean avgTemp generally makes sense for where we are located

	prevDay	prevWeek	avgTemp	wF	MonMed	classSession
count	3034.000000	3028.000000	3094.000000	3035.000000	3094.000000	3094.000000
mean	78.608096	78.636982	70.554299	78.599382	82.136933	0.604719
std	50.591626	50.635127	14.474940	50.585565	41.708586	0.488990
min	0.037854	0.037854	21.500000	0.037854	5.659191	0.000000
25%	36.084438	36.065511	60.000000	36.093901	51.103059	0.000000
50%	75.708236	75.708236	72.500000	75.708236	76.389610	1.000000
75%	121.133177	121.133177	83.000000	121.133177	117.082787	1.000000
max	188.210674	188.210674	96.000000	188.210674	170.305676	1.000000

Info

info: summarizes the metadata of a dataframe

Note: the term “metadata” refers to the data about a set of data. For example, the metadata of a rain depth sensor would be the type of sensor and where it was installed, and the data itself would be the rainfall measurements. In Python, the metadata refers to attributes of the dataframe that are not the data itself.

```
1  water_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3094 entries, 0 to 3093
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   allDate        3094 non-null   object
 1   prevDay        3034 non-null   float64
 2   avgTemp        3094 non-null   float64
 3   Water Usage    3035 non-null   float64
 4   MonMed         3094 non-null   float64
 5   classSession   3094 non-null   int64
 6   dayType        3094 non-null   object
 7   season         3094 non-null   object
 8   Twos           3094 non-null   float64
dtypes: float64(5), int64(1), object(3)
memory usage: 217.7+ KB
```

Editing a df: Rename a column, Remove a column, Add a new column

Renaming a column in a Dataframe

Existing variable name

New variable name

```
1 #Rename the column
2 water_df = water_df.rename(columns = {'wF':'Water Usage'})
3
4 #Show the first 3 rows to see the different column name
5 water_df.head(3)
```

	allDate	prevDay	prevWeek	avgTemp	Water Usage	MonMed	classSession	dayType	season
0	1/14/2009 0:00	NaN	NaN	47.0	7.192282	10.788424	0	wk	Winter
1	1/15/2009 0:00	7.192282	NaN	46.0	6.813741	10.788424	0	wk	Winter
2	1/16/2009 0:00	6.813741	NaN	41.5	7.570824	10.788424	0	wk	Winter

Removing a column in a Dataframe

Name of the column to be dropped

This specifies that we're talking about a column, not a row

```
1  #Removing a column
2  water_df=water_df.drop(['prevWeek'],axis=1)
3
4  #Taking a look at what we did. See! No prevWeek column anymore!
5  water_df.head(3)
6
```

	allDate	prevDay	avgTemp	Water Usage	MonMed	classSession	dayType	season
0	1/14/2009 0:00	NaN	47.0	7.192282	10.788424	0	wk	Winter
1	1/15/2009 0:00	7.192282	46.0	6.813741	10.788424	0	wk	Winter
2	1/16/2009 0:00	6.813741	41.5	7.570824	10.788424	0	wk	Winter

The **head** function is useful for checking your work as you go. It's a good idea to use it to print a few lines of your dataframe each time you make a change to it to make sure you actually did what you were trying to do.

Adding a column

Dataframe that already exists

Name of new column

Whatever value you want to put in the new column. In this case, each value is 2

```
2 water_df['Twos'] = np.ones((3094,1))*2
3
4 #Taking a look at what we did.
5 water_df.head(3)
```

	allDate	prevDay	avgTemp	Water Usage	MonMed	classSession	dayType	season	Twos
0	1/14/2009 0:00	NaN	47.0	7.192282	10.788424	0	wk	Winter	2.0
1	1/15/2009 0:00	7.192282	46.0	6.813741	10.788424	0	wk	Winter	2.0
2	1/16/2009 0:00	6.813741	41.5	7.570824	10.788424	0	wk	Winter	2.0

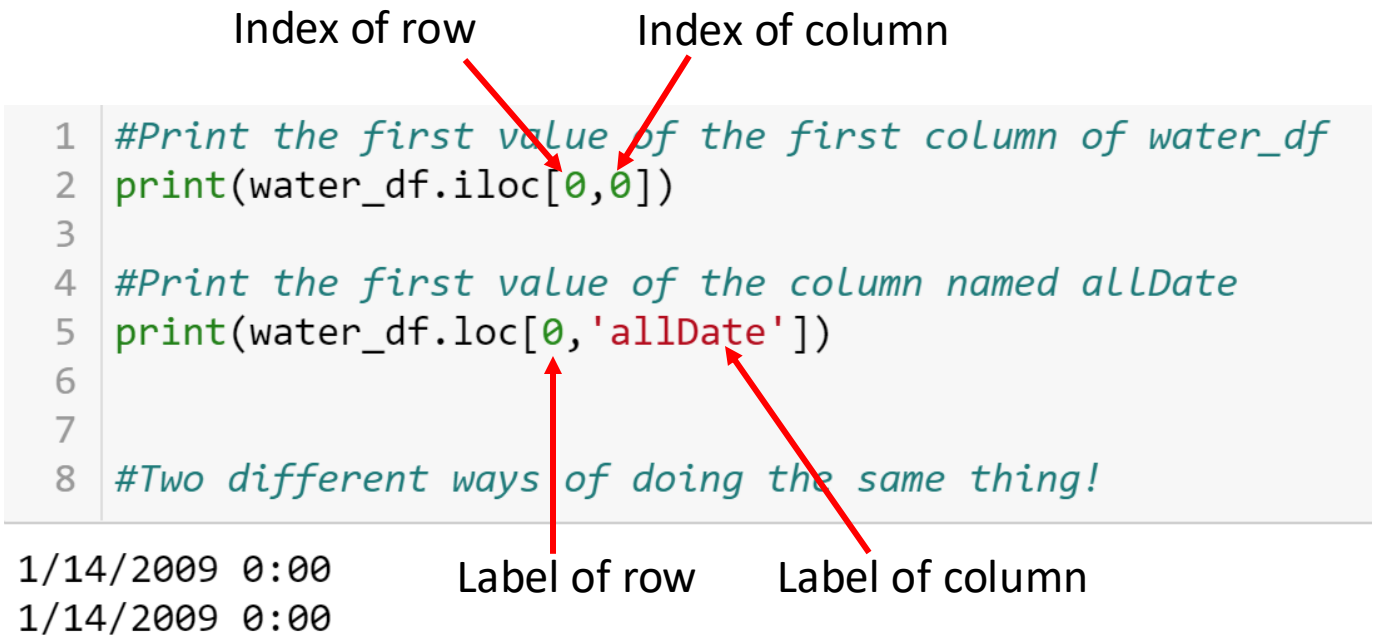
Note: whatever values are inserted into the dataframe should be 1 column, and the **same number of rows** as already exist in the dataframe

How to slice particular rows of a df?

Indexing a dataframe: 2 methods

iloc: refers to the indices of the row and column. Similar to the type of indexing we have already learned about arrays and lists

loc: refers to the labels of the rows and columns. By default, each row has the label of a number starting from 0, so loc is most useful for referencing by column name



How does pandas helps speed up statistical calculations?

df.groupby('Column Name")

```
water_df.groupby('dayType').mean(numeric_only=True)
```

✓ 0.0s

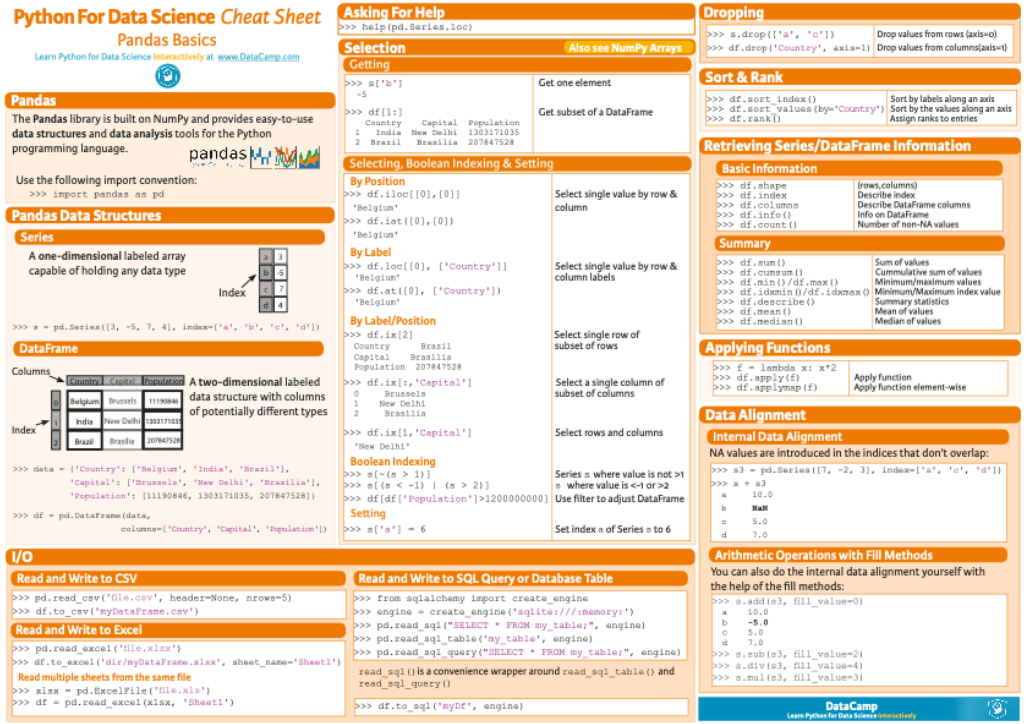
dayType	prevDay	avgTemp	Water Usage	MonMed	classSession	Twos
wk	86.467549	70.637104	91.275022	82.264107	0.614932	2.0
wknd	59.431741	70.347285	47.756281	81.818999	0.579186	2.0

Averaged columns for Weekdays

Averaged columns for Weekends

Some notes on the lab activity

- Lab 3 dataset and Pandas Cheat Sheet in Canvas
- **Import 'JES_Water_Lab3.csv'**



*** Python file extensions**

.ipynb = jupyter

.py = SPYDER

In-class activity

- Work through Lab 3 Student Activity
- Make sure you can complete the activity before taking Lab 3 Quiz. The quiz will ask you some questions about the activity.
- Lab Quiz 3 due September 19, 11:59 pm

Recap

- Distinction between arrays and dataframes
- Size and Shape
- Summarizing a dataframe
- Renaming, removing, and adding columns of a dataframe
- Indexing a dataframe, .groupby()